Model Comparison Report

Walmart Weekly Sales Forecasting

A write-up of how the five models stack up against each other, why XGBoost was picked, and what the model is actually paying attention to when it makes a prediction.

1. Comparing the Models

Five models went through the same pipeline and the same 80/20 hold-out split: Linear Regression, a base Random Forest, a tuned Random Forest (first with RandomizedSearchCV, then re-tuned again with Optuna), XGBoost, and an ARIMA model fit on the store-aggregated weekly series. Below is how they came out.

1.1 Performance Table

Model	MAE	RMSE	R^2 Score
XGBoost	1,387.36	3,317.46	0.9789
Random Forest (Tuned)	1,487.48	3,993.69	0.9694
Random Forest (Base)	1,636.23	4,596.05	0.9595
Linear Regression	2,402.11	6,344.95	0.9228
ARIMA (aggregate series)	3,597,509.82	3,924,134.12	-4.14

XGBoost comes out on top on every metric that's actually usable here — lowest MAE, lowest RMSE, and the highest R^2. The tuned Random Forest is a clear second, and both forests comfortably beat plain Linear Regression.

The ARIMA row needs a caveat: it isn't forecasting the same thing as the other four. It's trained on total sales added up across every store and department, so the error is on a completely different scale and shouldn't be read side-by-side with the row-level numbers above it. Its negative R^2 just means it does worse than guessing the average — worth saying outright rather than quietly leaving it in the table without comment.

MAPE was also calculated for the four row-level models, and honestly, the numbers aren't usable as they came out:

Model	MAPE (as computed)
Linear Regression	8.66×10^{14}
Random Forest	2.93×10^{13}
Random Forest (Tuned)	3.50×10^{13}
XGBoost	2.01×10^{14}

1.2 Trade-offs, Not Just Accuracy

Accuracy alone puts the models in this order: XGBoost, then tuned Random Forest, then base Random Forest, then Linear Regression. But a real deployment decision weighs more than accuracy:

Model	Accuracy	Interpretability	Training Cost	Deployment
Linear Regression	Lowest of the four	Highest — coefficients read directly	Trivial, fits almost instantly	Simplest — a linear equation
Random Forest (base)	Mid-range	Low — 10 trees are hard to inspect individually	Low, only 10 estimators	Simple; larger model file than linear regression
Random Forest (Tuned)	Second-best	Same as above, plus a search step to explain	Highest — RandomizedSearchCV, then a second Optuna study on top	Same as base RF, but retraining means re-running the search
XGBoost	Best	Lower than linear regression, but SHAP recovers a lot of it	Moderate — 300 boosted trees, no hyperparameter search run here	Needs the xgboost runtime; a bit more moving parts than sklearn-only models

Training time itself wasn't actually timed in the notebook — the "training cost" column above is a judgment call based on what each step does (how many estimators, whether a search runs, how many trials), not a stopwatch. Adding a couple of time.time() calls around each .fit() would turn that into a real, measured column instead of an estimate.

Taking accuracy and everything else together, XGBoost is the right call whenever getting the prediction as close as possible matters most, and the interpretability gap can be closed afterward with SHAP. Linear Regression still earns its place as the transparent fallback — the one model where a store manager could look at the coefficients and understand the whole thing without any extra tooling.

1.3 Business Context

- Real-time prediction: none of these models are running live — everything here is a batch prediction over a hold-out set. XGBoost summing 300 trees per row is still comfortably fast enough for weekly batch forecasting at the store-department level; it would only start to matter for a genuinely low-latency use case.
- Cost of errors: XGBoost's MAE of about 1,387 is the lowest of the group, which matters at scale — across roughly 3,000+ store-department combinations, that difference adds up to a lot less capital sitting in unnecessary safety stock.
- Inventory planning: the SHAP results later in this report show the model leans heavily on recent sales history (last week's number, the last month's average) — which lines up with how reorder points already get set in practice, so the model's behavior is easy to sanity-check against what the business already knows.
- Promotions: markdown activity does show up among the more influential features, backing up the pattern already visible in the EDA — markdowns genuinely move demand, which is useful to know when timing future promotions.

1.4 Where This Falls Short, and What I'd Fix Next

- ARIMA here is really just a baseline sanity check on the overall seasonal pattern, not a usable store-department forecaster — its negative R^2 is a real limitation, not a rounding error.
- Random Forest was tuned twice, but both searches (RandomizedSearchCV and Optuna) only sampled 2 parameter combinations each. That's enough to beat the untuned model, but nowhere near enough to say the forest has been properly optimized.
- MAPE isn't trustworthy as reported, and WMAE — arguably the metric that matters most for this problem — isn't computed at all yet.

- The train/test split for the four row-level models is a random shuffle, not a time-aware split. More on why that matters in the next section — it's the biggest thing I'd change if I revisited this.
- Next step, in priority order: switch to a proper time-based split, add WMAE to the results table, and actually time each model's training run instead of estimating it.

2. Evaluation & Validation

2.1 Metrics Used, and Why

Metric	Why it's here
MAE	Reads directly in Weekly_Sales units, so it's the easiest number to explain to someone non-technical: "off by about $1,387 per store-department-week, on average."
RMSE	Squares the errors before averaging, so a single badly-missed holiday spike costs more than a string of small misses — which matches how costly a big miss actually is to the business.
WMAE	Not computed here yet, but worth calling out: this is what the original Kaggle competition actually scores on, weighting holiday weeks 5x. Given how much holiday timing matters for this business, it's probably the most relevant metric of all and the clearest gap in the current evaluation.
MAPE	Meant to give a percentage error that's comparable across stores and departments of very different sizes — undermined here by the near-zero sales weeks discussed above.
R^2	A quick gut-check on how much of the week-to-week variance is being explained versus just predicting the average — most useful alongside MAE/RMSE, not by itself.

2.2 How the Split Was Done

The four row-level models were all evaluated on one random 80/20 split, created with `train_test_split(..., shuffle=True, random_state=42)`. ARIMA is the one exception — it's evaluated on an actual chronological hold-out, the last 20% of dates in the aggregate series, which is the correct way to test a time-series model.

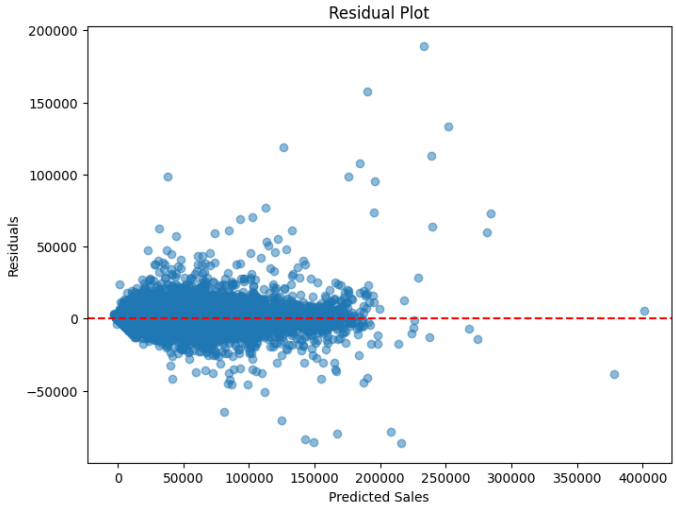
This is worth being upfront about: shuffling the split means a store-department's later weeks can land in training while an earlier week from that same series lands in the test set. Since features like Lag_1 and Rolling_Mean_4 are built directly from that time series, there's a real chance the model is picking up information adjacent in time to what it's being tested on — which tends to make hold-out performance look a little better than it would on genuinely unseen future weeks. TimeSeriesSplit is already imported in the notebook, but it only ever gets used inside the hyperparameter search, never for the actual train/test evaluation.

So the comparison between the four models is fair — they were all evaluated the same way — but the absolute numbers are probably a bit optimistic compared to what any of them would do forecasting real future weeks. A rolling-origin split (train on weeks 1 through N, test on N+1 onward, and step forward) would give a more honest read, even if the resulting MAE/RMSE come out a little worse than what's shown here.

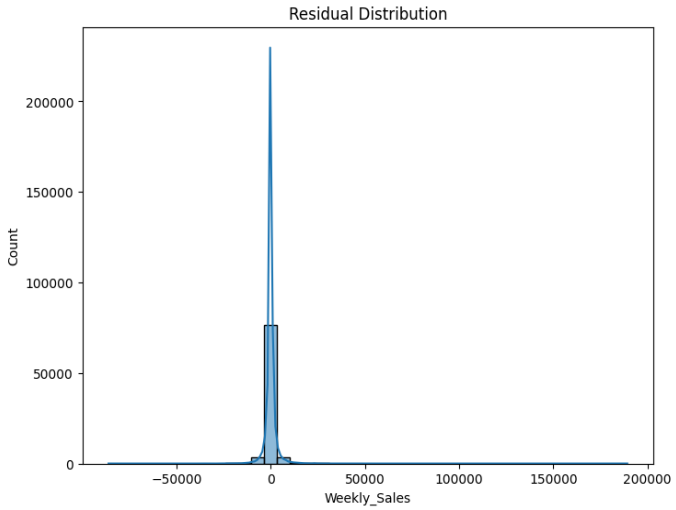
2.3 Looking at the Residuals

Three diagnostic plots came out of the XGBoost run, since it's the model actually going forward:

Actual vs. predicted sales, with the 45° reference line. Points track the line closely at lower sales volumes; the spread widens higher up, meaning the biggest misses happen on the biggest-selling weeks — which is consistent with RMSE being pulled up by a small number of large errors.



Residuals vs. predicted value. Mostly centered around zero without an obvious funnel shape, though a few large positive residuals show up at higher predictions — the model still under-predicts some of the bigger spikes.



Distribution of residuals. Roughly bell-shaped and centered near zero, which is a reasonable sign, aside from the heavier tail already visible in the plot above.

These three checks were only run for XGBoost. It'd be worth repeating the same three plots for Linear Regression and both Random Forest versions too, just to see whether the same pattern (fine in the middle, worse at the extremes) shows up across the board or is specific to XGBoost.

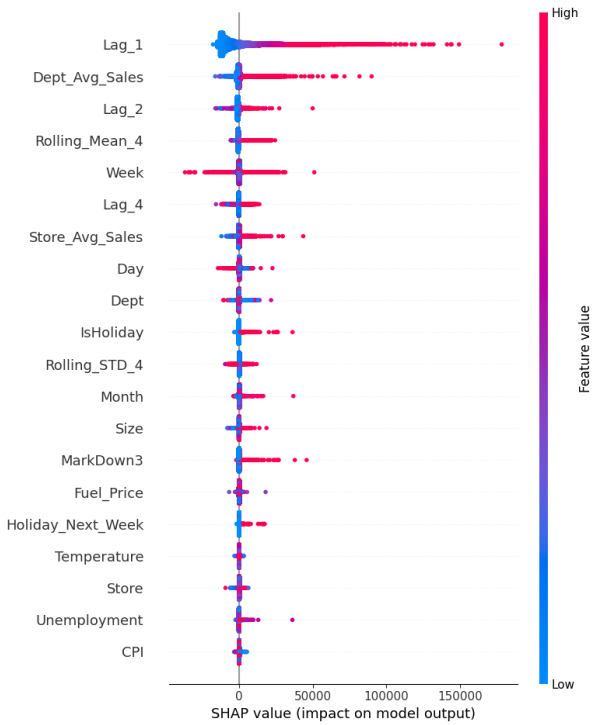
2.4 Breaking Performance Down by Segment

One thing this evaluation doesn't do yet is check whether the model's error changes depending on the situation — holiday week vs. not, store type A/B/C, whether a markdown was active. The EDA earlier in the project shows the raw sales differences across those groups, but that's different from checking the model's error across those same groups. That'd mean grouping the hold-out residuals (y_test minus the XGBoost prediction) by IsHoliday, by store type, and by markdown activity, and comparing MAE/RMSE within each group — a few extra groupby calls on predictions that already exist, nothing that needs retraining.

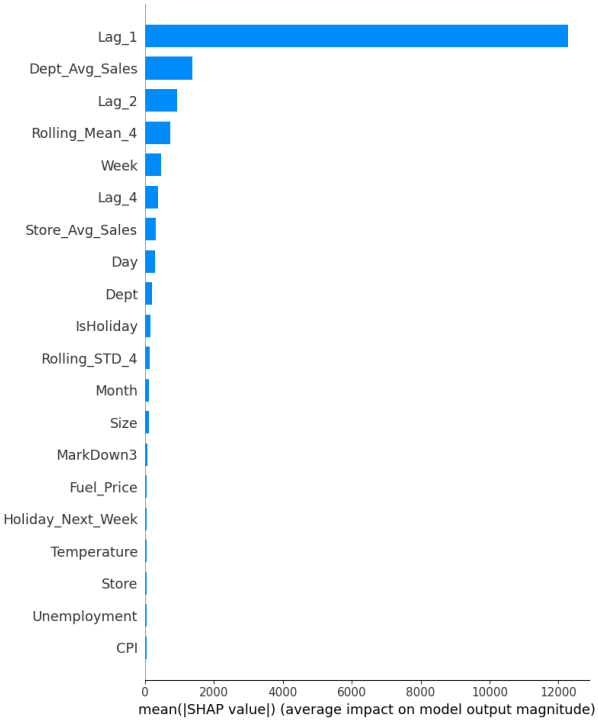
3. Explaining the Model

Everything in this section is built on the XGBoost model, since that's the one that won out — using SHAP's TreeExplainer, plus XGBoost's own built-in importance scores as a cross-check.

3.1 What the Model Is Paying Attention To



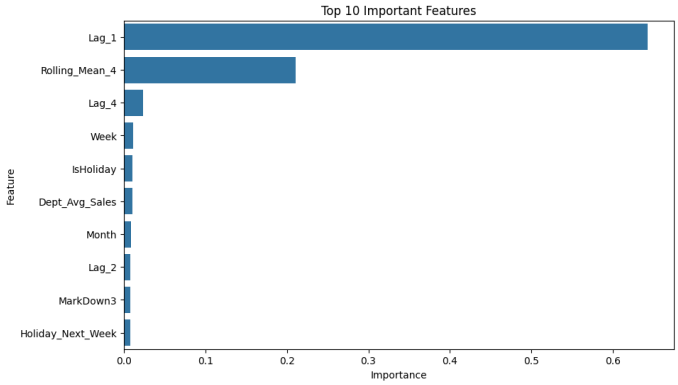
SHAP summary plot — each dot is one prediction; color shows whether the feature's value was high or low, and position shows how much it pushed the prediction up or down.



SHAP bar plot — the average size of each feature's impact, giving one overall ranking.

Both SHAP views and XGBoost's own feature_importances_ land on the same story: recent sales history dominates everything else.

Feature	XGBoost Importance
Lag_1 (previous week's sales)	0.6429
Rolling_Mean_4 (4-week rolling average)	0.2106
Lag_4 (sales 4 weeks prior)	0.0232
Week	0.0109
IsHoliday	0.0106
Dept_Avg_Sales	0.0104
Month	0.0085
Lag_2	0.0077
MarkDown3	0.0075
Holiday_Next_Week	0.0074



Top 10 features by XGBoost importance, matching the SHAP ranking above.

Lag_1 and Rolling_Mean_4 alone make up around 85% of everything the model relies on. Calendar effects, holiday flags, department averages, and markdowns are all real but clearly secondary. Worth saying plainly: this model is, at its core, a well-built momentum predictor. It's very good at carrying a recent trend one more week forward, and comparatively weak at reacting to anything that isn't already showing up in the last month of sales.

3.2 Explaining Individual Predictions

What's here so far is all global — it explains the model's behavior on average, across every prediction. It doesn't yet include a look at individual predictions (a waterfall or force plot for a handful of specific rows, showing exactly why one particular forecast came out the way it did). That's a small addition on top of what's already built, since the SHAP explainer object already exists — running it on three or four individual test rows (one clear over-prediction, one under-prediction, one that landed close to actual) would round this out nicely.

3.3 What This Means for the Business

- What drives sales, according to the model: overwhelmingly, recent momentum — what a store-department sold last week and over the past month — with week-of-year, holiday timing, department baseline, and markdowns all playing smaller supporting roles.
- Holiday prep: IsHoliday and Holiday_Next_Week both register as meaningful features, and the sales trend already shows holiday weeks running higher — so building in extra buffer ahead of a holiday is a pattern the model has genuinely picked up on, not just a business assumption.
- Markdowns: MarkDown3 shows up in the top 10 features, backing up the earlier observation that markdowns move demand — a reason to keep tracking markdown data carefully for planning purposes.

3.4 Fairness Across Stores

One more open question: does the model perform equally well across different store types and sizes, or is it quietly better for some stores than others? That hasn't been checked yet. The EDA shows how raw sales differ by store type, but not how the model's error differs by store type. Splitting the hold-out residuals by Type and by Size (maybe grouped into small/medium/large) and comparing MAE/RMSE across those groups would answer this directly — worth doing before treating this model as ready to use uniformly across every store.

Summary

XGBoost is the clear winner here on every metric worth trusting, and its errors hold up reasonably well under inspection. That part of the analysis is solid. What's left isn't a better model — it's a handful of follow-up checks that would make this a genuinely complete evaluation rather than just a model comparison:

- Add WMAE as a real metric instead of leaning on a MAPE calculation that doesn't hold up on this data.
- Move to an actual time-based validation split, since the current random shuffle is probably making every model look a little better than it would forecasting real future weeks.
- Break error down by holiday week, store type, and markdown activity, not just in aggregate.
- Add a few individual SHAP explanations alongside the global summary that's already here.
- Check whether the model is equally fair across different store types and sizes.

None of this needs a retrain — every one of these builds directly on the model, predictions, and SHAP explainer that already exist from this run.